트랜잭션 풀 메모리 최적화 분석

개념적 배경

geth의 트랜잭션 풀은 제한된 메모리 환경에서 최대한 많은 트랜잭션을 효율적으로 관리해야 합니다. 메모리 최적화는 노드의 안정성과 성능에 직접적인 영향을 미치므로, 다양한 기법을 통해 메모리 사용량을 제어하고 최적화합니다.

메모리 최적화가 필요한 이유

1. **리소스 제약**: 노드 운영 환경의 메모리 한계
2. **DoS 공격 방어**: 메모리 고갈 공격으로부터 보호
3. **성능 유지**: 메모리 압박 시에도 안정적인 성능 보장
4. **확장성**: 네트워크 성장에 따른 트랜잭션 증가 대응

핵심 최적화 전략

- **슬롯 기반 관리**: 트랜잭션 크기를 슬롯 단위로 정규화
- **계층적 제한**: 계정별, 전역별 다단계 제한
- **동적 정리**: 상황에 따른 적응적 메모리 정리
- **캐싱 최적화**: 중복 계산 방지와 메모리 재사용

구현 분석

1. 슬롯 기반 메모리 관리

1.1 슬롯 계산 메커니즘

위치: `core/txpool/legacypool/legacypool.go:1916`

```go
const (
txSlotSize = 32 * 1024  // 32KB per slot
txMaxSize = 4 * txSlotSize  // 128KB maximum transaction size
)

// numSlots calculates the number of slots needed for a single transaction.
func  numSlots(tx *types.Transaction) int {
return  int((tx.Size() + txSlotSize - 1) / txSlotSize)
}
```

슬롯 계산 공식:

```
slots = ⌈transaction_size / 32KB⌉
```

예시:

- 1KB 트랜잭션: 1 슬롯
- 50KB 트랜잭션: 2 슬롯
- 100KB 트랜잭션: 4 슬롯

1.2 슬롯 추적 시스템

위치: `core/txpool/legacypool/legacypool.go:1763`

```go
type lookup struct {
slots  int  // 총 사용 슬롯 수
```

```
lock sync.RWMutex // 동시성 제어
txs  map[common.Hash]*types.Transaction // 트랜잭션 저장소
auths  map[common.Address][]common.Hash // 권한 추적
}

// Add adds a transaction to the lookup.
func (t *lookup) Add(tx *types.Transaction) {
t.lock.Lock()
defer  t.lock.Unlock()

t.slots += numSlots(tx)
slotsGauge.Update(int64(t.slots))
t.txs[tx.Hash()] = tx
t.addAuthorities(tx)
}

// Remove removes a transaction from the lookup.
func (t *lookup) Remove(hash common.Hash) {
t.lock.Lock()
defer  t.lock.Unlock()

tx, ok := t.txs[hash]
if !ok {
```

```
return
}
t.slots -= numSlots(tx)
slotsGauge.Update(int64(t.slots))
delete(t.txs, hash)
t.removeAuthorities(tx)
}
```

2. 계층적 메모리 제한

2.1 설정 구조

위치: `core/txpool/legacypool/legacypool.go:140`

```
type Config struct {
AccountSlots  uint64  // 계정당 Pending 슬롯 보장
GlobalSlots  uint64  // 전체 Pending 슬롯 한도
AccountQueue  uint64  // 계정당 Queued 슬롯 한도
GlobalQueue  uint64  // 전체 Queued 슬롯 한도
Lifetime time.Duration // 트랜잭션 생존 시간
}

// 기본 설정값
var  DefaultConfig = Config{
AccountSlots: 16, // 계정당 16 슬롯 (512KB)
```

```
GlobalSlots: 5120, // 전체 5120 슬롯 (160MB)
AccountQueue: 64, // 계정당 64 슬롯 (2MB)
GlobalQueue: 1024, // 전체 1024 슬롯 (32MB)
Lifetime: 3 * time.Hour, // 3시간
}
```

2.2 메모리 사용량 계산

총 메모리 사용량:

```
Total Memory = (GlobalSlots + GlobalQueue) × 32KB
= (5120 + 1024) × 32KB
= 196.6MB (최대)
```

계정별 메모리 할당:

```
Per Account = (AccountSlots + AccountQueue) × 32KB
= (16 + 64) × 32KB
= 2.56MB (최대)
```

3. 동적 메모리 정리 시스템

3.1 Pending 큐 정리 (truncatePending)

위치: `core/txpool/legacypool/legacypool.go:1510`

```
func (pool *LegacyPool) truncatePending() {
```

```
pending := uint64(0)

// 1. 스팸 계정 식별 (AccountSlots 초과 계정)

spammers := prque.New[uint64, common.Address](nil)

for  addr, list := range  pool.pending {

length := uint64(list.Len())

pending += length

if  length > pool.config.AccountSlots {

spammers.Push(addr, length)

}

}

// 2. 전역 한도 확인

if  pending <= pool.config.GlobalSlots {

return

}

// 3. 균등화 알고리즘으로 점진적 제거

offenders := []common.Address{}

for  pending > pool.config.GlobalSlots && !spammers.Empty() {

offender, _ := spammers.Pop()

offenders = append(offenders, offender)

// 모든 위반자의 트랜잭션 수를 균등하게 조정

if  len(offenders) > 1 {

threshold := pool.pending[offender].Len()

// 임계값까지 모든 위반자 감소
```

```
for pending > pool.config.GlobalSlots &&
pool.pending[offenders[len(offenders)-2]].Len() > threshold {
for i := 0; i < len(offenders)-1; i++ {
list := pool.pending[offenders[i]]
caps := list.Cap(list.Len() - 1)
for _, tx := range caps {
pool.all.Remove(tx.Hash())
pool.pendingNonces.setIfLower(offenders[i], tx.Nonce())
}
pending--
}
}
}
}
}
```

균등화 알고리즘 특징:

- 가장 많은 트랜잭션을 가진 계정부터 처리
- 모든 위반자의 트랜잭션 수를 점진적으로 균등화
- 공정성을 보장하면서 메모리 압박 해소

3.2 Queued 큐 정리 (truncateQueue)

위치: `core/txpool/legacypool/legacypool.go:1600`

```
func (pool *LegacyPool) truncateQueue() {
queued := uint64(0)
for _, list := range pool.queue {
queued += uint64(list.Len())
}
if queued <= pool.config.GlobalQueue {
return
}
// 하트비트 기준 계정 정렬 (오래된 순)
addresses := make(addressesByHeartbeat, 0, len(pool.queue))
for addr := range pool.queue {
addresses = append(addresses, addressByHeartbeat{
addr, pool.beats[addr]
})
}
sort.Sort(sort.Reverse(addresses))
// 오래된 계정부터 트랜잭션 제거
for drop := queued - pool.config.GlobalQueue; drop > 0 && len(addresses) > 0;
addr := addresses[len(addresses)-1]
list := pool.queue[addr.address]
addresses = addresses[:len(addresses)-1]
if size := uint64(list.Len()); size <= drop {
```

```
// 전체 계정 트랜잭션 제거
for _, tx := range list.Flatten() {
pool.removeTx(tx.Hash(), true, true)
}
drop -= size
} else {
// 높은 논스부터 부분 제거
txs := list.Flatten()
for i := len(txs) - 1; i >= 0 && drop > 0; i-- {
pool.removeTx(txs[i].Hash(), true, true)
drop--
}
}
}
}
```

4. 가격 기반 제거 시스템

4.1 이중 힙 구조

위치: `core/txpool/legacypool/list.go:527`

```
type pricedList struct {
stales atomic.Int64 // 오래된 항목 수
all *lookup // 전체 트랜잭션 참조
```

```
urgent priceHeap // 긴급 힙 (effective tip 기준)

floating priceHeap // 부유 힙 (gas fee cap 기준)

reheapMu sync.Mutex // 재정렬 뮤텍스

}


const (

urgentRatio = 4  // 긴급 힙 비율

floatingRatio = 1  // 부유 힙 비율

)
```

4.2 저가 트랜잭션 제거 (Discard)

```
func (l *pricedList) Discard(slots  int) (types.Transactions, bool) {

drop := make(types.Transactions, 0, slots)

for  slots > 0 {

// 힙 크기 비율에 따라 제거 전략 결정

if  len(l.urgent.list)*floatingRatio > len(l.floating.list)*urgentRatio {

// Urgent에서 Floating으로 강등

tx := heap.Pop(&l.urgent).(*types.Transaction)

if  l.all.Get(tx.Hash()) != nil {

heap.Push(&l.floating, tx)

}

} else {

// Floating에서 완전 제거
```

```
tx := heap.Pop(&l.floating).(*types.Transaction)
if l.all.Get(tx.Hash()) != nil {
drop = append(drop, tx)
slots -= numSlots(tx)
}
}
}
return drop, true
}
```

5. 시간 기반 정리 시스템

5.1 주기적 정리 루프

위치: `core/txpool/legacypool/legacypool.go:337`

```
var (
evictionInterval = time.Minute // 정리 주기
)

func (pool *LegacyPool) loop() {
evict := time.NewTicker(evictionInterval)
defer evict.Stop()
for {
select {
```

```
case <-evict.C:
pool.mu.Lock()
for addr := range pool.queue {
// 생존 시간 초과 트랜잭션 제거
if time.Since(pool.beats[addr]) > pool.config.Lifetime {
list := pool.queue[addr].Flatten()
for _, tx := range list {
pool.removeTx(tx.Hash(), true, true)
}
queuedEvictionMeter.Mark(int64(len(list)))
}
}
pool.mu.Unlock()
}
}
}
```

5.2 하트비트 추적

```
type addressByHeartbeat struct {
address common.Address
heartbeat time.Time
}
```

```go
// 트랜잭션 추가 시 하트비트 업데이트
func (pool *LegacyPool) add(tx *types.Transaction) {
from, _ := types.Sender(pool.signer, tx)
pool.beats[from] = time.Now() // 하트비트 갱신
}
```

6. 캐싱 최적화

6.1 정렬 결과 캐싱

위치: `core/txpool/legacypool/list.go:200`

```go
type SortedMap struct {
items  map[uint64]*types.Transaction
index *nonceHeap
cache types.Transactions // 정렬 결과 캐시
cacheMu sync.Mutex
}

func (m *SortedMap) flatten() types.Transactions {
m.cacheMu.Lock()
defer  m.cacheMu.Unlock()
// 캐시가 없으면 생성
if  m.cache == nil {
```

```
m.cache = make(types.Transactions, 0, len(m.items))
for _, tx := range m.items {
m.cache = append(m.cache, tx)
}
sort.Sort(types.TxByNonce(m.cache))
}
return m.cache
}

// 변경 시 캐시 무효화
func (m *SortedMap) Put(tx *types.Transaction) {
m.cacheMu.Lock()
m.items[tx.Nonce()], m.cache = tx, nil // 캐시 무효화
m.cacheMu.Unlock()
}
```

6.2 힙 재구성 최적화

```
func (l *pricedList) Removed(count int) {
stales := l.stales.Add(int64(count))
// 25% 이상 오래된 항목이 있을 때만 재구성
if int(stales) <= (len(l.urgent.list)+len(l.floating.list))/4 {
return
}
```

```
l.Reheap()
}


func (l *pricedList) Reheap() {
l.reheapMu.Lock()
defer  l.reheapMu.Unlock()
start := time.Now()
l.stales.Store(0)
// 전체 힙 재구성
l.urgent.list = make([]*types.Transaction, 0, l.all.Count())
l.all.Range(func(hash common.Hash, tx *types.Transaction) bool {
l.urgent.list = append(l.urgent.list, tx)
return true
})
heap.Init(&l.urgent)
// 균형 조정
floatingCount := len(l.urgent.list) * floatingRatio / (urgentRatio + floatingRat
l.floating.list = make([]*types.Transaction, floatingCount)
for  i := 0;  i < floatingCount;  i++ {
l.floating.list[i] = heap.Pop(&l.urgent).(*types.Transaction)
}
heap.Init(&l.floating)
reheapTimer.Update(time.Since(start))
```

```
}
```

실행 흐름

1. 메모리 압박 감지 및 대응

```
graph TD

A[트랜잭션 추가] --> B{전역 한도 확인}

B -->|초과| C[저가 트랜잭션 확인]

C -->|있음| D[가격 기반 제거]

C -->|없음| E[트랜잭션 거부]

B -->|여유| F[풀에 추가]

D --> G{공간 확보됨?}

G -->|Yes| F

G -->|No| E

F --> H[슬롯 카운터 업데이트]

H --> I[메트릭 갱신]
```

2. 주기적 정리 과정

```
graph TD

A[정리 타이머 발동] --> B[Queued 큐 검사]

B --> C{생존 시간 초과?}
```

```
C -->|Yes|  D[트랜잭션 제거]

C -->|No|  E[다음 계정]

D -->  F[메모리 해제]

F -->  G[메트릭 업데이트]

G -->  E

E -->  H{모든 계정 처리?}

H -->|No|  B

H -->|Yes|  I[정리 완료]
```

성능 최적화

1. 메모리 할당 최적화

```go
// 미리 할당된 슬라이스 사용

func (pool *LegacyPool) truncatePending() {

// 예상 크기로 미리 할당

offenders := make([]common.Address, 0, 16)

// 재사용 가능한 버퍼 사용

for  pending > pool.config.GlobalSlots {

// 메모리 할당 최소화

}

}
```

2. 락 경합 최소화

```
// 읽기 락과 쓰기 락 분리
func (t *lookup) Get(hash common.Hash) *types.Transaction {
t.lock.RLock() // 읽기 락만 사용
defer t.lock.RUnlock()
return t.txs[hash]
}

// 배치 처리로 락 획득 횟수 감소
func (pool *LegacyPool) addTxsLocked(txs []*types.Transaction) {
// 한 번의 락으로 여러 트랜잭션 처리
for _, tx := range txs {
pool.add(tx)
}
}
```

3. 가비지 컬렉션 압박 감소

```
// 객체 재사용
var txSlicePool = sync.Pool{
New: func() interface{} {
return make([]*types.Transaction, 0, 128)
```

```
},

}


func getTxSlice() []*types.Transaction {

return txSlicePool.Get().([]*types.Transaction)[:0]

}


func putTxSlice(slice []*types.Transaction) {

txSlicePool.Put(slice)

}
```

실제 사용 예시

1. 메모리 사용량 모니터링

```
// 현재 슬롯 사용량 확인

totalSlots := pool.all.Slots()

pendingSlots := 0

queuedSlots := 0


for _, list := range pool.pending {

for _, tx := range list.Flatten() {

pendingSlots += numSlots(tx)
```

```
}

}


for _, list := range pool.queue {

for _, tx := range list.Flatten() {

queuedSlots += numSlots(tx)

}

}


fmt.Printf("Total: %d, Pending: %d, Queued: %d\n",

totalSlots, pendingSlots, queuedSlots)
```

2. 설정 튜닝

```
// 메모리 제약 환경을 위한 설정

config := legacypool.DefaultConfig

config.GlobalSlots = 2048  // 64MB로 감소

config.GlobalQueue = 512  // 16MB로 감소

config.AccountSlots = 8  // 계정당 256KB로 감소

config.AccountQueue = 32  // 계정당 1MB로 감소

config.Lifetime = time.Hour  // 생존 시간 단축


pool := legacypool.New(config, blockchain)
```

3. 메모리 압박 상황 처리

```
// 메모리 압박 시 강제 정리
func forceCleanup(pool *legacypool.LegacyPool) {
// 낮은 가격 트랜잭션 제거
threshold := big.NewInt(1000000000) // 1 Gwei
lowPriceTxs := pool.all.TxsBelowTip(threshold)
for _, tx := range lowPriceTxs {
pool.removeTx(tx.Hash(), true, true)
}
// 강제 힙 재구성
pool.priced.Reheap()
}
```

모니터링 및 메트릭

주요 메트릭

```
// 메모리 관련 메트릭
var (
slotsGauge = metrics.NewRegisteredGauge("txpool/slots", nil)
pendingGauge = metrics.NewRegisteredGauge("txpool/pending", nil)
queuedGauge = metrics.NewRegisteredGauge("txpool/queued", nil)
```

```
// 제거 관련 메트릭

pendingRateLimitMeter = metrics.NewRegisteredMeter("txpool/pending/ratelimit", n

queuedEvictionMeter = metrics.NewRegisteredMeter("txpool/queued/eviction", nil)

// 성능 메트릭

reheapTimer = metrics.NewRegisteredTimer("txpool/reheap", nil)

)
```

알림 임계값

```
// 메모리 사용량 알림

func checkMemoryUsage(pool *legacypool.LegacyPool) {

slots := pool.all.Slots()

maxSlots := pool.config.GlobalSlots + pool.config.GlobalQueue

usage := float64(slots) / float64(maxSlots)

if usage > 0.9 {

log.Warn("Transaction pool memory usage high", "usage", usage)

} else if usage > 0.8 {

log.Info("Transaction pool memory usage elevated", "usage", usage)

}

}
```

관련 시스템

의존성 관계

- **가비지 컬렉터**: 메모리 해제와 GC 압박 관리
- **메트릭 시스템**: 메모리 사용량 모니터링
- **설정 시스템**: 동적 한도 조정
- **로깅 시스템**: 메모리 관련 이벤트 기록

상호작용 패턴

1. **적응적 제한**: 네트워크 상황에 따른 동적 한도 조정
2. **예측적 정리**: 메모리 압박 전 사전 정리
3. **우선순위 기반**: 중요도에 따른 선별적 보존
4. **배치 최적화**: 대량 작업의 효율적 처리

영향 범위

- **노드 안정성**: 메모리 고갈 방지로 안정성 향상
- **처리 성능**: 효율적인 메모리 사용으로 성능 최적화
- **네트워크 참여**: 리소스 제약 환경에서도 노드 운영 가능
- **확장성**: 트랜잭션 증가에 대한 적응적 대응

이 메모리 최적화 시스템은 제한된 리소스 환경에서도 안정적이고 효율적인 트랜잭션 풀 운영을 가능하게 하는 핵심 메커니즘입니다.